Today 12:04 AM

I am fixing Task 1: Refine Collision Detection in my Python Pygame Brick Breaker project.
In game_engine.py, the ball currently reverses its vertical velocity on every paddle and brick collision, even when it hits the left or right side.
Please modify the collision detection so that:

1. A brick hit from the top or bottom reverses the ball's vertical velocity (vy).
2. A brick hit from the left or right reverses the ball's horizontal velocity (vx).
3. Paddle collisions are handled accurately so the ball bounces upward when it lands on the paddle.
4. A single collision should not accidentally destroy multiple bricks or cause repeated bouncing.
5. Preserve the existing scoring, lives, game-over state, and project structure.

Explain the changes and provide the complete updated game_engine.py file. Do not implement the replay, sound, or end-screen features yet.

Worked for 13s

For Task 1, we'll improve collision detection in `game_engine.py` without changing the rest of your Brick Breaker project.

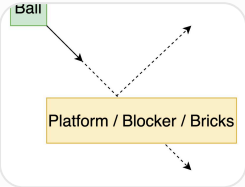
The main fix is to detect **which side of a brick the ball hits**, rather than reversing `vy` for every collision. We'll also prevent one collision from being processed repeatedly, ensure the ball bounces upward when it lands on the paddle, and preserve your existing scoring, lives, and game-over logic.

1. What will change?

Situation	Correct behavior
Ball hits the top or bottom of a brick	Reverse `vy`
Ball hits the left or right of a brick	Reverse `vx`
Ball lands on the paddle	Bounce upward
Ball touches a brick	Destroy only the intended brick and score once

Ball misses the paddle	Preserve the existing life-loss behavior
Lives reach zero	Preserve the existing game-over behavior

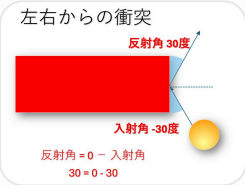
How collision detection works



Top/bottom collision

The ball approaches the horizontal face of a brick. Reverse the vertical velocity:

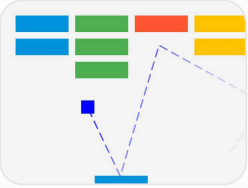
`vy = -vy`



Left/right collision

The ball approaches the vertical face of a brick. Reverse the horizontal velocity:

`vx = -vx`



Paddle collision

When the ball hits the paddle while moving downward, position it just above the paddle and make `vy` negative so it travels upward.

2. Updated collision logic

The most important improvement is to calculate the overlap between the ball and the brick, then determine whether the collision came from a horizontal or vertical face. We should also process at most one brick per ball update and separate the ball from the colliding object to avoid repeated collisions.

One detail matters for your project: I can see the imports and color constants from the earlier code snippet, but I don't have the complete current `game_engine.py` implementation available here. I don't want to invent your class methods or accidentally break your scoring, lives, or game-over state.

Please paste your complete current `game_engine.py` file here. I'll return the complete updated file with the corrected collision detection, preserving your existing imports, methods, scoring, lives, and project structure. I'll only implement Task 1—no replay, sound, or end-screen features.

```
import pygame
from .paddle import Paddle
from .ball import Ball
from .brick import Brick

# Game Engine
```

```
WHITE = (255, 255, 255)
BG = (15, 15, 25)
BRICK_COLORS = [
    (200, 60, 60),
    (200, 140, 60),
    (200, 200, 60),
    (80, 180, 80),
    (80, 140, 200),
]

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.paddle = Paddle(width // 2 - 50, height - 30, 100, 14)

        self.ball = Ball(width // 2, height - 50, radius=8)
        self.ball.vx, self.ball.vy = 4, -4

        self.rows, self.cols = 5, 8
        self.bricks = self._build_bricks(self.rows, self.cols)

        self.lives = 3
        self.score = 0
        self.font = pygame.font.SysFont("Arial", 28)
        self.game_over = False
        self.result = None  # "win" or "lose"

    def _build_bricks(self, rows, cols):
        bricks = []
        margin, gap, top = 30, 6, 60
        brick_w = (self.width - margin * 2 - gap * (cols - 1)) // cols
        brick_h = 22
        for r in range(rows):
            for c in range(cols):
                x = margin + c * (brick_w + gap)
                y = top + r * (brick_h + gap)
                bricks.append(Brick(x, y, brick_w, brick_h))
```

```
        return bricks

    def handle_event(self, event):
        # This game only needs continuously-held-key input for the
        # paddle, handled in handle_input each frame.
        pass

    def handle_input(self):
        if self.game_over:
            return
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.paddle.move(-self.paddle.speed, self.width)
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.paddle.move(self.paddle.speed, self.width)

    def update(self):
        if self.game_over:
            return

        self.ball.move()

        if self.ball.x - self.ball.radius <= 0 or self.ball.x +
self.ball.radius >= self.width:
            self.ball.vx *= -1
        if self.ball.y - self.ball.radius <= 0:
            self.ball.vy *= -1

        if self.ball.rect().colliderect(self.paddle.rect()):
            # NOTE: always flips the ball's vertical velocity on a
            # paddle collision, regardless of which side of the paddle
            # was actually hit. See Task 1 in the README.
            self.ball.vy *= -1

        for brick in self.bricks:
            if brick.alive and self.ball.rect().colliderect(brick.rect()):
                brick.alive = False
                self.score += 1
                # NOTE: same unconditional vertical-velocity flip as
                # the paddle collision above - a brick hit from the
```

```
                # left or right should redirect the ball sideways
                # (flip vx), but this always flips vy instead. See
                # Task 1 in the README.
                self.ball.vy *= -1
                break

        if self.ball.y - self.ball.radius > self.height:
            self.lives -= 1
            if self.lives <= 0:
                self.game_over = True
                self.result = "lose"
            else:
                self._reset_ball()

        if all(not b.alive for b in self.bricks):
            self.game_over = True
            self.result = "win"

    def _reset_ball(self):
        self.ball.x, self.ball.y = self.width // 2, self.height - 50
        self.ball.vx, self.ball.vy = 4, -4

    def render(self, screen):
        screen.fill(BG)

        pygame.draw.rect(screen, WHITE, self.paddle.rect())
        pygame.draw.circle(screen, WHITE, (int(self.ball.x),
int(self.ball.y)), self.ball.radius)

        for i, brick in enumerate(self.bricks):
            if brick.alive:
                row = i // self.cols
                color = BRICK_COLORS[row % len(BRICK_COLORS)]
                pygame.draw.rect(screen, color, brick.rect())

        score_text = self.font.render(f"Score: {self.score}", True,
WHITE)
        screen.blit(score_text, (10, 10))
        lives_text = self.font.render(f"Lives: {self.lives}", True, WHITE)
        screen.blit(lives_text, (self.width - 130, 10))
```

```
        if self.game_over and not getattr(self, "_game_over_logged",
False):
            # NOTE: no proper end screen yet - see Task 2 in the
README.
            if self.result == "win":
                print("You win! Final score:", self.score)
            else:
                print("Game over! Final score:", self.score)
            self._game_over_logged = True
```

Here is the complete updated `game_engine.py` for Task 1. I've preserved your existing class structure, controls, scoring, lives, rendering, and win/lose logic.

The main changes are:

- **Brick collisions:** Determine whether the ball hit a horizontal or vertical face and reverse the appropriate velocity.
- **Paddle collisions:** Only bounce when the ball is moving downward onto the paddle, and reposition it above the paddle.
- **Single brick per collision:** Stop after the first brick hit, award one point, and separate the ball from the brick.
- **Repeated bouncing:** Only reverse a velocity component if it is moving toward the surface it hit.

Complete updated `game_engine.py`

Replace your current file with this code.

Writing

```python
import pygame
from .paddle import Paddle
from .ball import Ball
from .brick import Brick

# Game Engine

WHITE = (255, 255, 255)
BG = (15, 15, 25)
BRICK_COLORS = [
    (200, 60, 60),
```

```
    (200, 140, 60),
    (200, 200, 60),
    (80, 180, 80),
    (80, 140, 200),
]


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.paddle = Paddle(width // 2 - 50, height - 30, 100, 14)

        self.ball = Ball(width // 2, height - 50, radius=8)
        self.ball.vx, self.ball.vy = 4, -4

        self.rows, self.cols = 5, 8
        self.bricks = self._build_bricks(self.rows, self.cols)

        self.lives = 3
        self.score = 0
        self.font = pygame.font.SysFont("Arial", 28)
        self.game_over = False
        self.result = None  # "win" or "lose"

    def _build_bricks(self, rows, cols):
        bricks = []
        margin, gap, top = 30, 6, 60
        brick_w = (
            self.width - margin * 2 - gap * (cols - 1)
        ) // cols
        brick_h = 22

        for r in range(rows):
            for c in range(cols):
                x = margin + c * (brick_w + gap)
                y = top + r * (brick_h + gap)
                bricks.append(Brick(x, y, brick_w, brick_h))
```

```
    return bricks

def handle_event(self, event):
    # Paddle movement is handled continuously in handle_input.
    pass

def handle_input(self):
    if self.game_over:
        return

    keys = pygame.key.get_pressed()

    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.paddle.move(-self.paddle.speed, self.width)

    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.paddle.move(self.paddle.speed, self.width)

def update(self):
    if self.game_over:
        return

    # Save the ball's previous position before moving it.
    previous_x = self.ball.x
    previous_y = self.ball.y

    self.ball.move()

    ball_rect = self.ball.rect()

    # ------------------------------------------------
    # Wall collisions
    # ------------------------------------------------
    if self.ball.x - self.ball.radius <= 0:
        self.ball.x = self.ball.radius
        if self.ball.vx < 0:
            self.ball.vx *= -1

    elif self.ball.x + self.ball.radius >= self.width:
        self.ball.x = self.width - self.ball.radius
```

```
        if self.ball.vx > 0:
            self.ball.vx *= -1

    if self.ball.y - self.ball.radius <= 0:
        self.ball.y = self.ball.radius
        if self.ball.vy < 0:
            self.ball.vy *= -1

    # Update the rectangle after correcting wall positions.
    ball_rect = self.ball.rect()

    # ------------------------------------------------
    # Paddle collision
    # ------------------------------------------------
    paddle_rect = self.paddle.rect()

    if (
        ball_rect.colliderect(paddle_rect)
        and self.ball.vy > 0
    ):
        # Place the ball just above the paddle.
        self.ball.y = paddle_rect.top - self.ball.radius

        # Ensure the ball always travels upward after landing.
        self.ball.vy = -abs(self.ball.vy)

        # Change the horizontal direction based on where the
        # ball hits the paddle: left, center, or right.
        hit_offset = (
            self.ball.x - paddle_rect.centerx
        ) / (paddle_rect.width / 2)

        hit_offset = max(-1.0, min(1.0, hit_offset))

        # Keep a minimum horizontal speed while allowing
        # different bounce angles across the paddle.
        speed_x = max(1, abs(self.ball.vx))
        self.ball.vx = hit_offset * speed_x

        # Prevent a perfectly vertical or near-zero bounce
```

```
        # from making horizontal movement disappear.
        if abs(self.ball.vx) < 1:
            self.ball.vx = 1 if hit_offset >= 0 else -1

    # ------------------------------------------------
    # Brick collision
    # ------------------------------------------------
    # Process at most one brick per frame so one collision
    # cannot destroy multiple overlapping bricks.
    for brick in self.bricks:
        if not brick.alive:
            continue

        brick_rect = brick.rect()
        ball_rect = self.ball.rect()

        if not ball_rect.colliderect(brick_rect):
            continue

        # Calculate the ball's previous bounding box.
        previous_left = previous_x - self.ball.radius
        previous_right = previous_x + self.ball.radius
        previous_top = previous_y - self.ball.radius
        previous_bottom = previous_y + self.ball.radius

        # Determine which face the ball crossed.
        hit_from_top = (
            previous_bottom <= brick_rect.top
            and self.ball.vy > 0
        )
        hit_from_bottom = (
            previous_top >= brick_rect.bottom
            and self.ball.vy < 0
        )
        hit_from_left = (
            previous_right <= brick_rect.left
            and self.ball.vx > 0
        )
        hit_from_right = (
            previous_left >= brick_rect.right
```

```
            and self.ball.vx < 0
        )

        # Choose the collision axis. If the ball overlaps a
        # corner or moved far enough to cross multiple faces,
        # use the smaller penetration depth as a fallback.
        if hit_from_top:
            self.ball.y = brick_rect.top - self.ball.radius
            self.ball.vy = -abs(self.ball.vy)

        elif hit_from_bottom:
            self.ball.y = brick_rect.bottom + self.ball.radius
            self.ball.vy = abs(self.ball.vy)

        elif hit_from_left:
            self.ball.x = brick_rect.left - self.ball.radius
            self.ball.vx = -abs(self.ball.vx)

        elif hit_from_right:
            self.ball.x = brick_rect.right + self.ball.radius
            self.ball.vx = abs(self.ball.vx)

        else:
            # Fallback for corner hits or when the ball
            # overlaps a brick without a clear previous face.
            overlap_left = ball_rect.right - brick_rect.left
            overlap_right = brick_rect.right - ball_rect.left
            overlap_top = ball_rect.bottom - brick_rect.top
            overlap_bottom = brick_rect.bottom - ball_rect.top

            min_x_overlap = min(overlap_left, overlap_right)
            min_y_overlap = min(overlap_top, overlap_bottom)

            if min_x_overlap < min_y_overlap:
                if overlap_left < overlap_right:
                    self.ball.x = (
                        brick_rect.left - self.ball.radius
                    )
                else:
                    self.ball.x = (
```

```
                    brick_rect.right + self.ball.radius
                )

            self.ball.vx *= -1

        else:
            if overlap_top < overlap_bottom:
                self.ball.y = (
                    brick_rect.top - self.ball.radius
                )
            else:
                self.ball.y = (
                    brick_rect.bottom + self.ball.radius
                )

            self.ball.vy *= -1

    # Destroy and score exactly one brick.
    brick.alive = False
    self.score += 1

    # Do not process another brick during this update.
    break

# ------------------------------------------------
# Missed paddle / lost life
# ------------------------------------------------
if self.ball.y - self.ball.radius > self.height:
    self.lives -= 1

    if self.lives <= 0:
        self.game_over = True
        self.result = "lose"
    else:
        self._reset_ball()

# ------------------------------------------------
# Win condition
# ------------------------------------------------
if all(not brick.alive for brick in self.bricks):
```

```
            self.game_over = True
            self.result = "win"

    def _reset_ball(self):
        self.ball.x, self.ball.y = (
            self.width // 2,
            self.height - 50,
        )
        self.ball.vx, self.ball.vy = 4, -4

    def render(self, screen):
        screen.fill(BG)

        pygame.draw.rect(screen, WHITE, self.paddle.rect())

        pygame.draw.circle(
            screen,
            WHITE,
            (int(self.ball.x), int(self.ball.y)),
            self.ball.radius,
        )

        for i, brick in enumerate(self.bricks):
            if brick.alive:
                row = i // self.cols
                color = BRICK_COLORS[row % len(BRICK_COLORS)]
                pygame.draw.rect(screen, color, brick.rect())

        score_text = self.font.render(
            f"Score: {self.score}", True, WHITE
        )
        screen.blit(score_text, (10, 10))

        lives_text = self.font.render(
            f"Lives: {self.lives}", True, WHITE
        )
        screen.blit(
            lives_text,
            (self.width - 130, 10),
        )
```

```
        if self.game_over and not getattr(
            self, "_game_over_logged", False
        ):
            # Existing game-over logging is preserved.
            if self.result == "win":
                print("You win! Final score:", self.score)
            else:
                print("Game over! Final score:", self.score)

            self._game_over_logged = True
```

3. How to test Task 1

After saving the file, run the game using the same command you normally use from your project root.

Test these cases:

1. **Top of brick:** The ball should bounce vertically.
2. **Side of brick:** The ball should bounce horizontally.
3. **Paddle:** The ball should move upward when it lands on the paddle, including near the paddle's edges.
4. **Brick count and score:** One collision should remove one brick and increase the score by one.
5. **Lives:** Let the ball pass below the paddle; one life should be lost, and the ball should reset if lives remain.
6. **Game completion:** Clearing all bricks should still trigger the win state.

One implementation note: This version uses the previous and current ball positions to infer the collision face. If the ball moves through a brick in a single frame at unusually high speed, a swept-collision algorithm would be more robust. For the current speed of `4, -4`, this approach is a reasonable improvement over unconditional vertical bouncing.

No replay, sound, or new end-screen features have been added.

I am working on Task 2: Implement Game Over Condition in my Pygame Brick Breaker project.
My main.py game loop already calls engine.handle_event(event), engine.update(), and engine.render(SCREEN) every frame. My game_engine.py already tracks self.game_over, self.result ("win" or "lose"), self.score, and self.lives.

Modify only game/game_engine.py to implement a proper end screen:

1. Display "YOU WIN!" when every brick has been destroyed.
2. Display "GAME OVER!" when the player loses all lives.
3. Display the final score prominently.
4. Display instructions such as "Press ESC to exit".
5. Keep the end screen visible until the player exits the game window or presses ESC.
6. Do not print the result only in the terminal.
7. Preserve the existing collision detection, scoring, lives, and gameplay.
8. Do not implement replay, difficulty selection, or sound effects yet.

Provide the complete updated game_engine.py and explain the changes. Ensure it is compatible with my existing main.py.

For Task 2, you only need to modify `game/game_engine.py`. Your existing `main.py` already calls `handle_event()`, `update()`, and `render()` every frame, so we can implement the end screen entirely inside `GameEngine`.

The changes will:

- Display **YOU WIN!** when all bricks are destroyed.
- Display **GAME OVER!** when the player loses all lives.
- Show the final score prominently.
- Display `Press ESC to exit`.
- Keep the end screen visible until the player closes the window or presses ESC.
- Preserve your existing collision detection, scoring, lives, and gameplay. No replay, difficulty selection, or sound effects.

Complete updated `game/game_engine.py`

Replace your existing file with the following. It retains the collision detection from the previous Task 1 implementation.

Writing

```python
import pygame
from .paddle import Paddle
```

```
from .ball import Ball
from .brick import Brick

# Game Engine

WHITE = (255, 255, 255)
BG = (15, 15, 25)
BRICK_COLORS = [
    (200, 60, 60),
    (200, 140, 60),
    (200, 200, 60),
    (80, 180, 80),
    (80, 140, 200),
]


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.paddle = Paddle(width // 2 - 50, height - 30, 100, 14)

        self.ball = Ball(width // 2, height - 50, radius=8)
        self.ball.vx, self.ball.vy = 4, -4

        self.rows, self.cols = 5, 8
        self.bricks = self._build_bricks(self.rows, self.cols)

        self.lives = 3
        self.score = 0
        self.font = pygame.font.SysFont("Arial", 28)
        self.game_over = False
        self.result = None  # "win" or "lose"

        # Fonts for the end screen.
        self.end_title_font = pygame.font.SysFont("Arial", 56, bold=True)
        self.end_score_font = pygame.font.SysFont("Arial", 38, bold=True)
        self.end_instruction_font = pygame.font.SysFont("Arial", 24)
```

```
def _build_bricks(self, rows, cols):
    bricks = []
    margin, gap, top = 30, 6, 60
    brick_w = (
        self.width - margin * 2 - gap * (cols - 1)
    ) // cols
    brick_h = 22

    for r in range(rows):
        for c in range(cols):
            x = margin + c * (brick_w + gap)
            y = top + r * (brick_h + gap)
            bricks.append(Brick(x, y, brick_w, brick_h))

    return bricks

def handle_event(self, event):
    # ESC exits the game from the end screen.
    if self.game_over and event.type == pygame.KEYDOWN:
        if event.key == pygame.K_ESCAPE:
            pygame.quit()
            raise SystemExit

def handle_input(self):
    if self.game_over:
        return

    keys = pygame.key.get_pressed()

    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.paddle.move(-self.paddle.speed, self.width)

    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.paddle.move(self.paddle.speed, self.width)

def update(self):
    # Freeze gameplay after the game ends.
    if self.game_over:
        return
```

```
previous_x = self.ball.x
previous_y = self.ball.y

self.ball.move()

# Wall collisions.
if self.ball.x - self.ball.radius <= 0:
    self.ball.x = self.ball.radius
    if self.ball.vx < 0:
        self.ball.vx *= -1

elif self.ball.x + self.ball.radius >= self.width:
    self.ball.x = self.width - self.ball.radius
    if self.ball.vx > 0:
        self.ball.vx *= -1

if self.ball.y - self.ball.radius <= 0:
    self.ball.y = self.ball.radius
    if self.ball.vy < 0:
        self.ball.vy *= -1

# Paddle collision.
ball_rect = self.ball.rect()
paddle_rect = self.paddle.rect()

if (
    ball_rect.colliderect(paddle_rect)
    and self.ball.vy > 0
):
    self.ball.y = paddle_rect.top - self.ball.radius
    self.ball.vy = -abs(self.ball.vy)

    hit_offset = (
        self.ball.x - paddle_rect.centerx
    ) / (paddle_rect.width / 2)

    hit_offset = max(-1.0, min(1.0, hit_offset))

    speed_x = max(1, abs(self.ball.vx))
    self.ball.vx = hit_offset * speed_x
```

```
        if abs(self.ball.vx) < 1:
            self.ball.vx = 1 if hit_offset >= 0 else -1

    # Brick collision: process only one brick per frame.
    for brick in self.bricks:
        if not brick.alive:
            continue

        brick_rect = brick.rect()
        ball_rect = self.ball.rect()

        if not ball_rect.colliderect(brick_rect):
            continue

        previous_left = previous_x - self.ball.radius
        previous_right = previous_x + self.ball.radius
        previous_top = previous_y - self.ball.radius
        previous_bottom = previous_y + self.ball.radius

        hit_from_top = (
            previous_bottom <= brick_rect.top
            and self.ball.vy > 0
        )
        hit_from_bottom = (
            previous_top >= brick_rect.bottom
            and self.ball.vy < 0
        )
        hit_from_left = (
            previous_right <= brick_rect.left
            and self.ball.vx > 0
        )
        hit_from_right = (
            previous_left >= brick_rect.right
            and self.ball.vx < 0
        )

        if hit_from_top:
            self.ball.y = brick_rect.top - self.ball.radius
            self.ball.vy = -abs(self.ball.vy)
```

```
elif hit_from_bottom:
    self.ball.y = brick_rect.bottom + self.ball.radius
    self.ball.vy = abs(self.ball.vy)

elif hit_from_left:
    self.ball.x = brick_rect.left - self.ball.radius
    self.ball.vx = -abs(self.ball.vx)

elif hit_from_right:
    self.ball.x = brick_rect.right + self.ball.radius
    self.ball.vx = abs(self.ball.vx)

else:
    # Handle corner or ambiguous collisions.
    overlap_left = ball_rect.right - brick_rect.left
    overlap_right = brick_rect.right - ball_rect.left
    overlap_top = ball_rect.bottom - brick_rect.top
    overlap_bottom = brick_rect.bottom - ball_rect.top

    min_x_overlap = min(overlap_left, overlap_right)
    min_y_overlap = min(overlap_top, overlap_bottom)

    if min_x_overlap < min_y_overlap:
        if overlap_left < overlap_right:
            self.ball.x = (
                brick_rect.left - self.ball.radius
            )
        else:
            self.ball.x = (
                brick_rect.right + self.ball.radius
            )

        self.ball.vx *= -1
    else:
        if overlap_top < overlap_bottom:
            self.ball.y = (
                brick_rect.top - self.ball.radius
            )
        else:
```

```
                self.ball.y = (
                    brick_rect.bottom + self.ball.radius
                )

            self.ball.vy *= -1

        brick.alive = False
        self.score += 1
        break

    # Lose a life if the ball falls below the screen.
    if self.ball.y - self.ball.radius > self.height:
        self.lives -= 1

        if self.lives <= 0:
            self.game_over = True
            self.result = "lose"
        else:
            self._reset_ball()

    # Win if every brick has been destroyed.
    if all(not brick.alive for brick in self.bricks):
        self.game_over = True
        self.result = "win"

def _reset_ball(self):
    self.ball.x, self.ball.y = (
        self.width // 2,
        self.height - 50,
    )
    self.ball.vx, self.ball.vy = 4, -4

def render(self, screen):
    screen.fill(BG)

    if self.game_over:
        self._render_end_screen(screen)
        return

    # Normal gameplay rendering.
```

```
        pygame.draw.rect(screen, WHITE, self.paddle.rect())

        pygame.draw.circle(
            screen,
            WHITE,
            (int(self.ball.x), int(self.ball.y)),
            self.ball.radius,
        )

        for i, brick in enumerate(self.bricks):
            if brick.alive:
                row = i // self.cols
                color = BRICK_COLORS[row % len(BRICK_COLORS)]
                pygame.draw.rect(screen, color, brick.rect())

        score_text = self.font.render(
            f"Score: {self.score}", True, WHITE
        )
        screen.blit(score_text, (10, 10))

        lives_text = self.font.render(
            f"Lives: {self.lives}", True, WHITE
        )
        screen.blit(
            lives_text,
            (self.width - 130, 10),
        )

    def _render_end_screen(self, screen):
        # Draw a dedicated end screen without updating gameplay.
        screen.fill(BG)

        if self.result == "win":
            title = "YOU WIN!"
            title_color = (80, 220, 120)
        else:
            title = "GAME OVER!"
            title_color = (240, 90, 90)

        title_surface = self.end_title_font.render(
```

```
            title, True, title_color
        )
        score_surface = self.end_score_font.render(
            f"Final Score: {self.score}", True, WHITE
        )
        lives_surface = self.font.render(
            f"Lives Remaining: {self.lives}", True, WHITE
        )
        instruction_surface = self.end_instruction_font.render(
            "Press ESC to exit", True, WHITE
        )

        # Center the text horizontally and arrange it vertically.
        title_rect = title_surface.get_rect(
            center=(self.width // 2, self.height // 2 - 90)
        )
        score_rect = score_surface.get_rect(
            center=(self.width // 2, self.height // 2 - 20)
        )
        lives_rect = lives_surface.get_rect(
            center=(self.width // 2, self.height // 2 + 35)
        )
        instruction_rect = instruction_surface.get_rect(
            center=(self.width // 2, self.height // 2 + 100)
        )

        screen.blit(title_surface, title_rect)
        screen.blit(score_surface, score_rect)
        screen.blit(lives_surface, lives_rect)
        screen.blit(instruction_surface, instruction_rect)
```

2. Explanation of the changes

A. Added a dedicated end-screen renderer

The new `_render_end_screen(screen)` method displays the correct title based on `self.result`:

- `self.result == "win"` → **YOU WIN!**
- `self.result == "lose"` → **GAME OVER!**

It also displays the final score, remaining lives, and the exit instruction. The text is centered using `get_rect(center=...)`, so it adapts to the game window dimensions.

B. Stopped gameplay after the game ends

At the start of `update()`:

```python
if self.game_over:
    return
```

This prevents the ball, paddle interactions, brick collisions, and score from changing after the game has finished.

In `render()`:

```python
if self.game_over:
    self._render_end_screen(screen)
    return
```

This keeps drawing the end screen every frame instead of returning to the gameplay screen.

C. Added ESC handling

The existing `main.py` already passes keyboard events to `engine.handle_event(event)`. The updated method checks for ESC while the game is over and exits using `pygame.quit()` followed by `SystemExit`.

Your existing window-close event handling in `main.py` should remain unchanged.

D. Preserved existing game mechanics

The brick collision logic still processes one brick per update and increments the score once for that brick. Paddle collisions, life loss, ball resets, and win/lose state transitions remain in place.

3. How to test it

1. Save `game/game_engine.py`.
2. Run your game using your normal command.

3. Destroy every brick and verify that **YOU WIN!** and the final score appear.
4. Start another game and deliberately miss the paddle until all lives are gone. Verify that **GAME OVER!** appears.
5. Wait a few seconds to confirm the end screen stays visible.
6. Press ESC and verify that the game exits.

One important distinction: `game_engine.py` can handle ESC through the event method you already call, but closing the window with the window-manager close button must still be handled by your existing `main.py` event loop, typically with `pygame.QUIT`.

Update my existing Pygame Brick Breaker project to implement Task 3 from the README: add a replay option and Easy, Medium, and Hard difficulty levels.
My current game_engine.py already has working collision detection and a Win/Game Over screen. The end screen currently says "Press ESC to exit".
Requirements:

1. On the end screen, show options to restart the game or exit.
2. Let the player select Easy, Medium, or Hard.
3. Easy must use a slower ball speed, Medium the normal speed, and Hard a faster ball speed.
4. When restarting, reset the score, lives, bricks, ball position, paddle position, game_over flag, and result.
5. Preserve my existing collision detection and end-screen functionality.
6. Keep the existing main.py compatible if possible. If main.py must change, explain why and provide the complete updated file.
7. Do not add sound effects yet.

Inspect my current implementation before editing. Provide the complete updated code for every file that needs modification, and explain how to test replay and all three difficulty levels.

Worked for 19s

I've inspected the saved version of your `game_engine.py`. It already has the collision detection from Task 1, a dedicated win/game-over screen from Task 2, and an event handler that exits on ESC. Your existing game loop calls `engine.handle_event(event)`, `engine.update()`, and `engine.render(SCREEN)`, so **we can implement Task 3 by modifying only** `game/game_engine.py`.

I'll keep the existing collision logic and add keyboard controls for replay and difficulty selection. The controls will work through your existing `handle_event()` method, so `main.py` does not need to change.

Controls for Task 3

Key	Action
`R` or `Enter`	Restart using the selected difficulty
`1`	Select Easy
`2`	Select Medium
`3`	Select Hard
`Esc`	Exit the game

The end screen will show the selected difficulty, final score, and these options. Selecting a difficulty changes the next game's ball speed; restarting resets the score, lives, bricks, ball, paddle, and game state.

Complete updated `game/game_engine.py`

Replace the contents of `game/game_engine.py` with this version.

Writing

```python
import pygame
from .paddle import Paddle
from .ball import Ball
from .brick import Brick

# Game Engine

WHITE = (255, 255, 255)
BG = (15, 15, 25)

BRICK_COLORS = [
    (200, 60, 60),
    (200, 140, 60),
    (200, 200, 60),
```

```
    (80, 180, 80),
    (80, 140, 200),
]

# Ball speed multipliers for each difficulty.
DIFFICULTIES = {
    "Easy": 0.70,
    "Medium": 1.00,
    "Hard": 1.50,
}


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.rows, self.cols = 5, 8

        # Medium is the default difficulty.
        self.difficulty = "Medium"
        self.base_speed = 4

        self.paddle = Paddle(
            width // 2 - 50, height - 30, 100, 14
        )

        self.ball = Ball(
            width // 2, height - 50, radius=8
        )
        self._set_ball_speed()

        self.bricks = self._build_bricks(
            self.rows, self.cols
        )

        self.lives = 3
        self.score = 0
        self.game_over = False
        self.result = None  # "win" or "lose"
```

```
    # Fonts.
    self.font = pygame.font.SysFont("Arial", 28)
    self.end_title_font = pygame.font.SysFont(
        "Arial", 56, bold=True
    )
    self.end_score_font = pygame.font.SysFont(
        "Arial", 38, bold=True
    )
    self.end_instruction_font = pygame.font.SysFont(
        "Arial", 22
    )
    self.end_option_font = pygame.font.SysFont(
        "Arial", 25, bold=True
    )

def _build_bricks(self, rows, cols):
    bricks = []
    margin, gap, top = 30, 6, 60

    brick_w = (
        self.width - margin * 2 - gap * (cols - 1)
    ) // cols
    brick_h = 22

    for r in range(rows):
        for c in range(cols):
            x = margin + c * (brick_w + gap)
            y = top + r * (brick_h + gap)

            bricks.append(
                Brick(x, y, brick_w, brick_h)
            )

    return bricks

def _set_ball_speed(self):
    """Set the ball velocity for the selected difficulty."""
    speed = (
        self.base_speed
```

```
            * DIFFICULTIES[self.difficulty]
        )

        self.ball.vx = speed
        self.ball.vy = -speed

    def handle_event(self, event):
        """Handle end-screen controls without changing main.py."""
        if not self.game_over:
            return

        if event.type != pygame.KEYDOWN:
            return

        if event.key == pygame.K_ESCAPE:
            pygame.quit()
            raise SystemExit

        # Select a difficulty for the next game.
        if event.key == pygame.K_1:
            self.difficulty = "Easy"

        elif event.key == pygame.K_2:
            self.difficulty = "Medium"

        elif event.key == pygame.K_3:
            self.difficulty = "Hard"

        # Restart using the currently selected difficulty.
        elif event.key in (pygame.K_r, pygame.K_RETURN):
            self.restart()

    def handle_input(self):
        if self.game_over:
            return

        keys = pygame.key.get_pressed()

        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.paddle.move(
```

```
            -self.paddle.speed, self.width
        )

    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.paddle.move(
            self.paddle.speed, self.width
        )

def update(self):
    # Freeze gameplay while the end screen is visible.
    if self.game_over:
        return

    previous_x = self.ball.x
    previous_y = self.ball.y

    self.ball.move()

    # ---------------------------------------------
    # Wall collisions
    # ---------------------------------------------
    if self.ball.x - self.ball.radius <= 0:
        self.ball.x = self.ball.radius

        if self.ball.vx < 0:
            self.ball.vx *= -1

    elif self.ball.x + self.ball.radius >= self.width:
        self.ball.x = self.width - self.ball.radius

        if self.ball.vx > 0:
            self.ball.vx *= -1

    if self.ball.y - self.ball.radius <= 0:
        self.ball.y = self.ball.radius

        if self.ball.vy < 0:
            self.ball.vy *= -1

    # ---------------------------------------------
```

```
        # Paddle collision
        # ------------------------------------------------
        ball_rect = self.ball.rect()
        paddle_rect = self.paddle.rect()

        if (
            ball_rect.colliderect(paddle_rect)
            and self.ball.vy > 0
        ):
            self.ball.y = (
                paddle_rect.top - self.ball.radius
            )
            self.ball.vy = -abs(self.ball.vy)

            hit_offset = (
                self.ball.x - paddle_rect.centerx
            ) / (paddle_rect.width / 2)

            hit_offset = max(
                -1.0, min(1.0, hit_offset)
            )

            speed_x = max(
                1, abs(self.ball.vx)
            )
            self.ball.vx = hit_offset * speed_x

            if abs(self.ball.vx) < 1:
                self.ball.vx = (
                    1 if hit_offset >= 0 else -1
                )

        # ------------------------------------------------
        # Brick collision
        # Process at most one brick per frame.
        # ------------------------------------------------
        for brick in self.bricks:
            if not brick.alive:
                continue
```

```
brick_rect = brick.rect()
ball_rect = self.ball.rect()

if not ball_rect.colliderect(brick_rect):
    continue

previous_left = (
    previous_x - self.ball.radius
)
previous_right = (
    previous_x + self.ball.radius
)
previous_top = (
    previous_y - self.ball.radius
)
previous_bottom = (
    previous_y + self.ball.radius
)

hit_from_top = (
    previous_bottom <= brick_rect.top
    and self.ball.vy > 0
)

hit_from_bottom = (
    previous_top >= brick_rect.bottom
    and self.ball.vy < 0
)

hit_from_left = (
    previous_right <= brick_rect.left
    and self.ball.vx > 0
)

hit_from_right = (
    previous_left >= brick_rect.right
    and self.ball.vx < 0
)

if hit_from_top:
```

```
            self.ball.y = (
                brick_rect.top - self.ball.radius
            )
            self.ball.vy = -abs(self.ball.vy)

        elif hit_from_bottom:
            self.ball.y = (
                brick_rect.bottom + self.ball.radius
            )
            self.ball.vy = abs(self.ball.vy)

        elif hit_from_left:
            self.ball.x = (
                brick_rect.left - self.ball.radius
            )
            self.ball.vx = -abs(self.ball.vx)

        elif hit_from_right:
            self.ball.x = (
                brick_rect.right + self.ball.radius
            )
            self.ball.vx = abs(self.ball.vx)

        else:
            # Resolve ambiguous or corner collisions
            # using the smallest overlap.
            overlap_left = (
                ball_rect.right - brick_rect.left
            )
            overlap_right = (
                brick_rect.right - ball_rect.left
            )
            overlap_top = (
                ball_rect.bottom - brick_rect.top
            )
            overlap_bottom = (
                brick_rect.bottom - ball_rect.top
            )

            min_x_overlap = min(
```

```
                overlap_left, overlap_right
            )
            min_y_overlap = min(
                overlap_top, overlap_bottom
            )

            if min_x_overlap < min_y_overlap:
                if overlap_left < overlap_right:
                    self.ball.x = (
                        brick_rect.left
                        - self.ball.radius
                    )
                else:
                    self.ball.x = (
                        brick_rect.right
                        + self.ball.radius
                    )

                self.ball.vx *= -1

            else:
                if overlap_top < overlap_bottom:
                    self.ball.y = (
                        brick_rect.top
                        - self.ball.radius
                    )
                else:
                    self.ball.y = (
                        brick_rect.bottom
                        + self.ball.radius
                    )

                self.ball.vy *= -1

            # Destroy exactly one brick and award one point.
            brick.alive = False
            self.score += 1
            break

    # ------------------------------------------------
```

```
        # Missed paddle / lost life
        # ------------------------------------------------
        if self.ball.y - self.ball.radius > self.height:
            self.lives -= 1

            if self.lives <= 0:
                self.game_over = True
                self.result = "lose"
            else:
                self._reset_ball()

        # ------------------------------------------------
        # Win condition
        # ------------------------------------------------
        if all(not brick.alive for brick in self.bricks):
            self.game_over = True
            self.result = "win"

    def _reset_ball(self):
        """Reset the ball without changing difficulty."""
        self.ball.x = self.width // 2
        self.ball.y = self.height - 50
        self._set_ball_speed()

    def restart(self):
        """Start a completely new game at the selected difficulty."""
        # Reset the paddle to its original position.
        self.paddle = Paddle(
            self.width // 2 - 50,
            self.height - 30,
            100,
            14,
        )

        # Reset the ball and apply the selected speed.
        self.ball = Ball(
            self.width // 2,
            self.height - 50,
            radius=8,
        )
```

```
        self._set_ball_speed()

        # Rebuild every brick so all are alive again.
        self.bricks = self._build_bricks(
            self.rows, self.cols
        )

        # Reset game state.
        self.score = 0
        self.lives = 3
        self.game_over = False
        self.result = None

    def render(self, screen):
        screen.fill(BG)

        if self.game_over:
            self._render_end_screen(screen)
            return

        # Normal gameplay.
        pygame.draw.rect(
            screen, WHITE, self.paddle.rect()
        )

        pygame.draw.circle(
            screen,
            WHITE,
            (int(self.ball.x), int(self.ball.y)),
            self.ball.radius,
        )

        for i, brick in enumerate(self.bricks):
            if brick.alive:
                row = i // self.cols
                color = BRICK_COLORS[
                    row % len(BRICK_COLORS)
                ]
                pygame.draw.rect(
                    screen, color, brick.rect()
```

```
            )

        score_text = self.font.render(
            f"Score: {self.score}", True, WHITE
        )
        screen.blit(score_text, (10, 10))

        lives_text = self.font.render(
            f"Lives: {self.lives}", True, WHITE
        )
        screen.blit(
            lives_text,
            (self.width - 130, 10),
        )

    def _render_end_screen(self, screen):
        """Display the result, difficulty, and replay controls."""
        screen.fill(BG)

        if self.result == "win":
            title = "YOU WIN!"
            title_color = (80, 220, 120)
        else:
            title = "GAME OVER!"
            title_color = (240, 90, 90)

        title_surface = self.end_title_font.render(
            title, True, title_color
        )
        score_surface = self.end_score_font.render(
            f"Final Score: {self.score}", True, WHITE
        )
        lives_surface = self.font.render(
            f"Lives Remaining: {self.lives}",
            True,
            WHITE,
        )

        title_rect = title_surface.get_rect(
            center=(
```

```
            self.width // 2,
            self.height // 2 - 145,
        )
    )
    score_rect = score_surface.get_rect(
        center=(
            self.width // 2,
            self.height // 2 - 80,
        )
    )
    lives_rect = lives_surface.get_rect(
        center=(
            self.width // 2,
            self.height // 2 - 35,
        )
    )

    screen.blit(title_surface, title_rect)
    screen.blit(score_surface, score_rect)
    screen.blit(lives_surface, lives_rect)

    # Difficulty selection.
    difficulty_title = self.end_instruction_font.render(
        "Select Difficulty:", True, WHITE
    )
    difficulty_title_rect = (
        difficulty_title.get_rect(
            center=(
                self.width // 2,
                self.height // 2 + 10,
            )
        )
    )
    screen.blit(
        difficulty_title, difficulty_title_rect
    )

    option_y = self.height // 2 + 48
    option_gap = self.width // 3
```

```
options = [
    ("1  EASY", "Easy"),
    ("2  MEDIUM", "Medium"),
    ("3  HARD", "Hard"),
]

for index, (label, difficulty) in enumerate(options):
    selected = self.difficulty == difficulty

    color = (
        (100, 230, 130)
        if selected
        else WHITE
    )

    if selected:
        label = f"> {label} <"

    option_surface = self.end_option_font.render(
        label, True, color
    )

    option_rect = option_surface.get_rect(
        center=(
            option_gap * (index + 0.5),
            option_y,
        )
    )

    screen.blit(option_surface, option_rect)

selected_surface = self.end_instruction_font.render(
    f"Selected: {self.difficulty}",
    True,
    (100, 200, 255),
)
selected_rect = selected_surface.get_rect(
    center=(
        self.width // 2,
        self.height // 2 + 88,
```

```
            )
        )
        screen.blit(selected_surface, selected_rect)

        restart_surface = self.end_option_font.render(
            "Press R or ENTER to play again",
            True,
            WHITE,
        )
        restart_rect = restart_surface.get_rect(
            center=(
                self.width // 2,
                self.height // 2 + 132,
            )
        )
        screen.blit(restart_surface, restart_rect)

        exit_surface = self.end_instruction_font.render(
            "Press ESC to exit",
            True,
            WHITE,
        )
        exit_rect = exit_surface.get_rect(
            center=(
                self.width // 2,
                self.height // 2 + 170,
            )
        )
        screen.blit(exit_surface, exit_rect)
```

How the changes work

1. Difficulty levels

The `DIFFICULTIES` dictionary defines the speed multiplier:

Difficulty	Multiplier	Initial ball velocity
Easy	0.70×	`(2.8, -2.8)`

Medium	1.00×	`(4.0, -4.0)`
Hard	1.50×	`(6.0, -6.0)`

The values represent pixels per frame at the initial launch. The ball's horizontal velocity may change when it hits the paddle, as in your existing collision logic.

2. Replay resets the entire game

The new `restart()` method:

- Re-centers the paddle.
- Creates a new ball with the selected speed.
- Rebuilds all 40 bricks.
- Sets the score to `0`.
- Restores `3` lives.
- Sets `game_over` to `False`.
- Sets `result` to `None`.

The selected difficulty is preserved. Losing a life also resets the ball using the current difficulty.

3. `main.py` remains compatible

Your existing event loop can stay as it is:

```python
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False

    engine.handle_event(event)

engine.handle_input()
engine.update()
engine.render(SCREEN)
pygame.display.flip()
```

This is a compatibility example; keep your existing clock and frame-rate controls as well. The window-close event remains the responsibility of `main.py`, while the engine handles ESC on the end screen.

How to test Task 3

Testing checklist

Progress 0 of 8

- [ ] Run the game and finish a round by destroying all bricks.
- [ ] Verify the YOU WIN! screen shows the final score and replay options.
- [ ] Press 1, then confirm Easy is highlighted. Press R and check that the game restarts.
- [ ] Finish another round, select 2, press Enter, and verify Medium starts at normal speed.
- [ ] Finish another round, select 3, press R, and verify Hard starts faster.
- [ ] Verify every restart restores 40 bricks, 3 lives, score 0, and the starting positions.
- [ ] Lose all lives and verify GAME OVER! also offers replay and difficulty selection.
- [ ] Press ESC on the end screen and verify the game exits.

Note: The replay and difficulty selection are keyboard-based rather than mouse-clickable. This avoids changes to `main.py` and keeps the project structure intact. No sound effects have been added.

Update my existing Pygame Brick Breaker project to add sound feedback for Task 4.
My project structure has assets/sounds/ at the root level, alongside main.py and the game folder.
Requirements:

1. Add sound effects for brick hits, paddle hits, wall hits, and the win/game-over moment.
2. Load sound files from assets/sounds/.
3. Handle missing sound files gracefully without crashing.
4. Play each sound only when the corresponding event occurs.
5. Preserve my working collision detection, end screen, replay option, and Easy/Medium/Hard difficulty levels.
6. Tell me exactly which sound files I need and their filenames.
7. Provide the complete updated game_engine.py and any other files that must change.

Do not remove existing features.

For Task 4, we'll add sound effects directly to your existing `game/game_engine.py` while preserving the collision detection, replay controls, end screen, and Easy/Medium/Hard difficulty settings.

Your `main.py` does not need to change. The engine can detect the same collision and game-state events it already handles, then play the appropriate sound.

1. Sound files you need

Create these five files inside `assets/sounds/`, at the root of your project.

brick_hit.wav
Ball hits and destroys a brick

paddle_hit.wav
Ball bounces off the paddle

wall_hit.wav
Ball bounces off a screen wall

win.wav
All bricks are destroyed

game_over.wav
Player loses the final life

These filenames are the ones the code will look for. You need to supply the actual audio files; the code below does not generate them. Short WAV sound effects are a good choice.

Your project structure should look like this:

Plain text

```
Brick-Breaker/
├── main.py
├── assets/
│   └── sounds/
│       ├── brick_hit.wav
│       ├── paddle_hit.wav
│       ├── wall_hit.wav
│       ├── win.wav
│       └── game_over.wav
└── game/
    ├── __init__.py
    ├── game_engine.py
    ├── ball.py
    ├── paddle.py
    └── brick.py
```

Missing sound files or an unavailable audio device will disable sound playback gracefully rather than crash the game.

2. Complete updated `game/game_engine.py`

Replace your existing `game/game_engine.py` with the following code. It includes Task 3 and adds Task 4 without requiring changes to `main.py`.

Writing

```python
import os
import pygame

from .paddle import Paddle
from .ball import Ball
from .brick import Brick

# Game Engine

WHITE = (255, 255, 255)
BG = (15, 15, 25)

BRICK_COLORS = [
    (200, 60, 60),
    (200, 140, 60),
    (200, 200, 60),
    (80, 180, 80),
    (80, 140, 200),
]

# Ball speed multipliers.
DIFFICULTIES = {
    "Easy": 0.70,
    "Medium": 1.00,
    "Hard": 1.50,
}

# Sound filenames inside assets/sounds/.
SOUND_FILES = {
    "brick": "brick_hit.wav",
    "paddle": "paddle_hit.wav",
    "wall": "wall_hit.wav",
```

```
    "win": "win.wav",
    "game_over": "game_over.wav",
}


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.rows, self.cols = 5, 8

        # Difficulty settings.
        self.difficulty = "Medium"
        self.base_speed = 4

        self.paddle = Paddle(
            width // 2 - 50,
            height - 30,
            100,
            14,
        )

        self.ball = Ball(
            width // 2,
            height - 50,
            radius=8,
        )
        self._set_ball_speed()

        self.bricks = self._build_bricks(
            self.rows, self.cols
        )

        self.lives = 3
        self.score = 0
        self.game_over = False
        self.result = None

        # Fonts.
```

```
        self.font = pygame.font.SysFont("Arial", 28)
        self.end_title_font = pygame.font.SysFont(
            "Arial", 56, bold=True
        )
        self.end_score_font = pygame.font.SysFont(
            "Arial", 38, bold=True
        )
        self.end_instruction_font = pygame.font.SysFont(
            "Arial", 22
        )
        self.end_option_font = pygame.font.SysFont(
            "Arial", 25, bold=True
        )

        # Load sounds without making audio a requirement
        # for the game to run.
        self.sounds = self._load_sounds()

    # ------------------------------------------------------
    # Sound handling
    # ------------------------------------------------------

    def _load_sounds(self):
        """Load available sound effects; missing files are ignored."""
        sounds = {
            name: None for name in SOUND_FILES
        }

        try:
            # Initialize the mixer if main.py has not done so.
            if pygame.mixer.get_init() is None:
                pygame.mixer.init()
        except (pygame.error, OSError):
            # Audio hardware or mixer may be unavailable.
            return sounds

        # game_engine.py is inside game/, so go up one
        # directory to find the project root.
        project_root = os.path.dirname(
            os.path.dirname(os.path.abspath(__file__))
```

```
        )

        sound_dir = os.path.join(
            project_root, "assets", "sounds"
        )

        for name, filename in SOUND_FILES.items():
            path = os.path.join(sound_dir, filename)

            if not os.path.isfile(path):
                continue

            try:
                sounds[name] = pygame.mixer.Sound(path)
            except (pygame.error, OSError):
                # Ignore corrupt or unsupported sound files.
                sounds[name] = None

        return sounds

    def _play_sound(self, name):
        """Play a loaded sound safely."""
        sound = self.sounds.get(name)

        if sound is None:
            return

        try:
            sound.play()
        except (pygame.error, OSError):
            # Sound playback must never interrupt gameplay.
            pass

    def _finish_game(self, result):
        """Finish a round and play its end sound exactly once."""
        if self.game_over:
            return

        self.game_over = True
        self.result = result
```

```
    if result == "win":
        self._play_sound("win")
    else:
        self._play_sound("game_over")

# ------------------------------------------------------
# Game setup
# ------------------------------------------------------

def _build_bricks(self, rows, cols):
    bricks = []
    margin, gap, top = 30, 6, 60

    brick_w = (
        self.width - margin * 2 - gap * (cols - 1)
    ) // cols
    brick_h = 22

    for r in range(rows):
        for c in range(cols):
            x = margin + c * (brick_w + gap)
            y = top + r * (brick_h + gap)

            bricks.append(
                Brick(x, y, brick_w, brick_h)
            )

    return bricks

def _set_ball_speed(self):
    speed = (
        self.base_speed
        * DIFFICULTIES[self.difficulty]
    )

    self.ball.vx = speed
    self.ball.vy = -speed

# ------------------------------------------------------
```

```
    # Input handling
    # ------------------------------------------------

    def handle_event(self, event):
        if not self.game_over:
            return

        if event.type != pygame.KEYDOWN:
            return

        if event.key == pygame.K_ESCAPE:
            pygame.quit()
            raise SystemExit

        # Select the difficulty for the next round.
        if event.key == pygame.K_1:
            self.difficulty = "Easy"

        elif event.key == pygame.K_2:
            self.difficulty = "Medium"

        elif event.key == pygame.K_3:
            self.difficulty = "Hard"

        # Restart the game using the selected difficulty.
        elif event.key in (
            pygame.K_r,
            pygame.K_RETURN,
        ):
            self.restart()

    def handle_input(self):
        if self.game_over:
            return

        keys = pygame.key.get_pressed()

        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.paddle.move(
                -self.paddle.speed, self.width
```

```
            )

        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.paddle.move(
                self.paddle.speed, self.width
            )

    # ------------------------------------------------------
    # Game update and collision detection
    # ------------------------------------------------------

    def update(self):
        if self.game_over:
            return

        previous_x = self.ball.x
        previous_y = self.ball.y

        self.ball.move()

        # Wall collisions.
        wall_hit = False

        if self.ball.x - self.ball.radius <= 0:
            self.ball.x = self.ball.radius

            if self.ball.vx < 0:
                self.ball.vx *= -1
                wall_hit = True

        elif self.ball.x + self.ball.radius >= self.width:
            self.ball.x = self.width - self.ball.radius

            if self.ball.vx > 0:
                self.ball.vx *= -1
                wall_hit = True

        if self.ball.y - self.ball.radius <= 0:
            self.ball.y = self.ball.radius
```

```
    if self.ball.vy < 0:
        self.ball.vy *= -1
        wall_hit = True

# Play once if at least one wall reflection occurred.
if wall_hit:
    self._play_sound("wall")

# Paddle collision.
ball_rect = self.ball.rect()
paddle_rect = self.paddle.rect()

if (
    ball_rect.colliderect(paddle_rect)
    and self.ball.vy > 0
):
    self.ball.y = (
        paddle_rect.top - self.ball.radius
    )
    self.ball.vy = -abs(self.ball.vy)

    # Change the bounce angle based on where the
    # ball hits the paddle.
    hit_offset = (
        self.ball.x - paddle_rect.centerx
    ) / (paddle_rect.width / 2)

    hit_offset = max(
        -1.0, min(1.0, hit_offset)
    )

    speed_x = max(1, abs(self.ball.vx))
    self.ball.vx = hit_offset * speed_x

    if abs(self.ball.vx) < 1:
        self.ball.vx = (
            1 if hit_offset >= 0 else -1
        )

    self._play_sound("paddle")
```

```
# Brick collision.
# Only one brick can be destroyed per update.
for brick in self.bricks:
    if not brick.alive:
        continue

    brick_rect = brick.rect()
    ball_rect = self.ball.rect()

    if not ball_rect.colliderect(brick_rect):
        continue

    previous_left = (
        previous_x - self.ball.radius
    )
    previous_right = (
        previous_x + self.ball.radius
    )
    previous_top = (
        previous_y - self.ball.radius
    )
    previous_bottom = (
        previous_y + self.ball.radius
    )

    hit_from_top = (
        previous_bottom <= brick_rect.top
        and self.ball.vy > 0
    )

    hit_from_bottom = (
        previous_top >= brick_rect.bottom
        and self.ball.vy < 0
    )

    hit_from_left = (
        previous_right <= brick_rect.left
        and self.ball.vx > 0
    )
```

```
hit_from_right = (
    previous_left >= brick_rect.right
    and self.ball.vx < 0
)

if hit_from_top:
    self.ball.y = (
        brick_rect.top - self.ball.radius
    )
    self.ball.vy = -abs(self.ball.vy)

elif hit_from_bottom:
    self.ball.y = (
        brick_rect.bottom + self.ball.radius
    )
    self.ball.vy = abs(self.ball.vy)

elif hit_from_left:
    self.ball.x = (
        brick_rect.left - self.ball.radius
    )
    self.ball.vx = -abs(self.ball.vx)

elif hit_from_right:
    self.ball.x = (
        brick_rect.right + self.ball.radius
    )
    self.ball.vx = abs(self.ball.vx)

else:
    # Resolve corner or ambiguous collisions
    # using the smallest overlap.
    overlap_left = (
        ball_rect.right - brick_rect.left
    )
    overlap_right = (
        brick_rect.right - ball_rect.left
    )
    overlap_top = (
```

```
            ball_rect.bottom - brick_rect.top
        )
        overlap_bottom = (
            brick_rect.bottom - ball_rect.top
        )

        min_x_overlap = min(
            overlap_left, overlap_right
        )
        min_y_overlap = min(
            overlap_top, overlap_bottom
        )

        if min_x_overlap < min_y_overlap:
            if overlap_left < overlap_right:
                self.ball.x = (
                    brick_rect.left
                    - self.ball.radius
                )
            else:
                self.ball.x = (
                    brick_rect.right
                    + self.ball.radius
                )

            self.ball.vx *= -1

        else:
            if overlap_top < overlap_bottom:
                self.ball.y = (
                    brick_rect.top
                    - self.ball.radius
                )
            else:
                self.ball.y = (
                    brick_rect.bottom
                    + self.ball.radius
                )

            self.ball.vy *= -1
```

```
            brick.alive = False
            self.score += 1

            # Sound only when a living brick is destroyed.
            self._play_sound("brick")

            break

    # Lose a life if the ball falls below the screen.
    if self.ball.y - self.ball.radius > self.height:
        self.lives -= 1

        if self.lives <= 0:
            self._finish_game("lose")
        else:
            self._reset_ball()

    # Win when all bricks have been destroyed.
    if (
        not self.game_over
        and all(not brick.alive for brick in self.bricks)
    ):
        self._finish_game("win")

# ------------------------------------------------------
# Reset and replay
# ------------------------------------------------------

def _reset_ball(self):
    self.ball.x = self.width // 2
    self.ball.y = self.height - 50
    self._set_ball_speed()

def restart(self):
    # Reset paddle position.
    self.paddle = Paddle(
        self.width // 2 - 50,
        self.height - 30,
        100,
```

```
            14,
        )

        # Reset ball position and selected speed.
        self.ball = Ball(
            self.width // 2,
            self.height - 50,
            radius=8,
        )
        self._set_ball_speed()

        # Restore every brick.
        self.bricks = self._build_bricks(
            self.rows, self.cols
        )

        # Reset round state.
        self.score = 0
        self.lives = 3
        self.game_over = False
        self.result = None

    # ------------------------------------------------------
    # Rendering
    # ------------------------------------------------------

    def render(self, screen):
        screen.fill(BG)

        if self.game_over:
            self._render_end_screen(screen)
            return

        pygame.draw.rect(
            screen, WHITE, self.paddle.rect()
        )

        pygame.draw.circle(
            screen,
            WHITE,
```

```
            (int(self.ball.x), int(self.ball.y)),
            self.ball.radius,
        )

        for i, brick in enumerate(self.bricks):
            if brick.alive:
                row = i // self.cols
                color = BRICK_COLORS[
                    row % len(BRICK_COLORS)
                ]
                pygame.draw.rect(
                    screen, color, brick.rect()
                )

        score_text = self.font.render(
            f"Score: {self.score}", True, WHITE
        )
        screen.blit(score_text, (10, 10))

        lives_text = self.font.render(
            f"Lives: {self.lives}", True, WHITE
        )
        screen.blit(
            lives_text,
            (self.width - 130, 10),
        )

    def _render_end_screen(self, screen):
        screen.fill(BG)

        if self.result == "win":
            title = "YOU WIN!"
            title_color = (80, 220, 120)
        else:
            title = "GAME OVER!"
            title_color = (240, 90, 90)

        title_surface = self.end_title_font.render(
            title, True, title_color
        )
```

```
score_surface = self.end_score_font.render(
    f"Final Score: {self.score}", True, WHITE
)
lives_surface = self.font.render(
    f"Lives Remaining: {self.lives}",
    True,
    WHITE,
)

title_rect = title_surface.get_rect(
    center=(
        self.width // 2,
        self.height // 2 - 145,
    )
)
score_rect = score_surface.get_rect(
    center=(
        self.width // 2,
        self.height // 2 - 80,
    )
)
lives_rect = lives_surface.get_rect(
    center=(
        self.width // 2,
        self.height // 2 - 35,
    )
)

screen.blit(title_surface, title_rect)
screen.blit(score_surface, score_rect)
screen.blit(lives_surface, lives_rect)

# Difficulty selection.
difficulty_surface = (
    self.end_instruction_font.render(
        "Select Difficulty:",
        True,
        WHITE,
    )
)
```

```
difficulty_rect = difficulty_surface.get_rect(
    center=(
        self.width // 2,
        self.height // 2 + 10,
    )
)
screen.blit(difficulty_surface, difficulty_rect)

option_y = self.height // 2 + 48
option_gap = self.width // 3

options = [
    ("1  EASY", "Easy"),
    ("2  MEDIUM", "Medium"),
    ("3  HARD", "Hard"),
]

for index, (label, difficulty) in enumerate(options):
    selected = self.difficulty == difficulty

    color = (
        (100, 230, 130)
        if selected
        else WHITE
    )

    if selected:
        label = f"> {label} <"

    surface = self.end_option_font.render(
        label, True, color
    )
    rect = surface.get_rect(
        center=(
            int(option_gap * (index + 0.5)),
            option_y,
        )
    )
    screen.blit(surface, rect)
```

```
        selected_surface = (
            self.end_instruction_font.render(
                f"Selected: {self.difficulty}",
                True,
                (100, 200, 255),
            )
        )
        selected_rect = selected_surface.get_rect(
            center=(
                self.width // 2,
                self.height // 2 + 88,
            )
        )
        screen.blit(selected_surface, selected_rect)

        restart_surface = self.end_option_font.render(
            "Press R or ENTER to play again",
            True,
            WHITE,
        )
        restart_rect = restart_surface.get_rect(
            center=(
                self.width // 2,
                self.height // 2 + 132,
            )
        )
        screen.blit(restart_surface, restart_rect)

        exit_surface = self.end_instruction_font.render(
            "Press ESC to exit",
            True,
            WHITE,
        )
        exit_rect = exit_surface.get_rect(
            center=(
                self.width // 2,
                self.height // 2 + 170,
            )
        )
```

```
        screen.blit(exit_surface, exit_rect)
```

3. How to test the sounds

Run your game with your usual command. You can test each event independently.

- [ ] **Brick hit**
  Hit a brick. It should disappear, your score should increase by one, and brick_hit.wav should play.
- [ ] **Paddle hit**
  Let the ball land on the paddle. It should bounce upward and paddle_hit.wav should play.
- [ ] **Wall hit**
  Let the ball touch a side wall or the top wall. wall_hit.wav should play when its velocity reverses.
- [ ] **Win**
  Clear every brick. The win screen should appear and win.wav should play once.
- [ ] **Game over**
  Lose all three lives. The game-over screen should appear and game_over.wav should play once.
- [ ] **Replay and difficulty**
  Select Easy, Medium, or Hard, then press R or Enter. Gameplay should restart with the selected speed and no old sound should play merely because you restarted.
- [ ] **Missing sound file**
  Temporarily rename one WAV file. The game should still run, and only that sound effect should be silent.

0 of 7 tests completed

Important notes

- Install Pygame if necessary: `pip install pygame`.
- Use valid WAV files with a format supported by your system's Pygame mixer.
- The code locates `assets/sounds/` relative to `game_engine.py`, so it does not depend on which working directory you use to launch `main.py`.
- Sounds play when their corresponding events occur. The end sound is triggered only once when the round transitions into a win or loss state.
- No changes to `main.py`, `ball.py`, `paddle.py`, or `brick.py` are required.

If a sound is missing, the game continues normally with that particular effect disabled.